

파이썬데이터분석



데이터사이언스 전공

담당교수: 서지훈



강남대학교

1. 파이썬 파일 처리(복습)

1.1 파일 열기: open 함수

- open 함수로 파일을 열 수 있으며 사용 후에는 close 함수로 닫아야 함
file=open(파일 이름, 모드)
- 다양한 옵션을 사용해 읽기, 쓰기 등의 모드 지정 가능

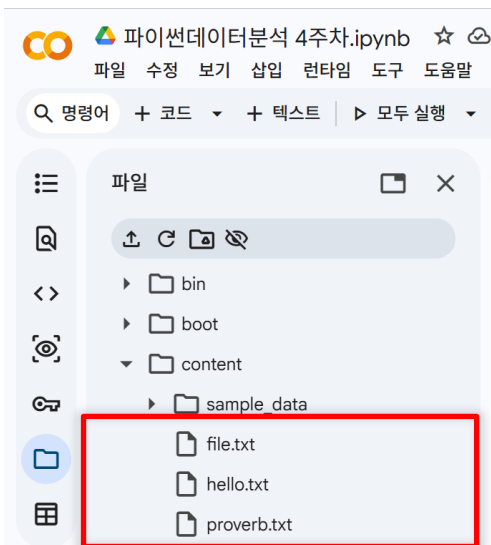
옵션	모드	설명
r	읽기 모드	읽기 모드로 파일을 연다.
w	쓰기 모드	파일에 내용을 쓰며 기존 내용은 사라진다. close 함수를 이용해 반드시 파일을 닫아야 변경 내용이 반영된다.
a	추가 모드	기존 파일에 내용이 추가된다. close 함수를 이용해 반드시 파일을 닫아야 변경 내용이 반영된다.
t	텍스트 모드	텍스트를 기록할 때 사용하며 기본값이다.
b	바이너리 모드	바이트 단위로 데이터를 기록할 때 사용한다.
x	쓰기 모드	파일 생성 후 쓰기 모드로 연다. 기존 파일이 있으면 오류가 발생한다.

1. 파이썬 파일 처리(복습)

1.2 open 함수 모드 예시들

- open 모드에 +를 포함하여 기존 파일 내용을 읽고 쓰는 방식 변경 가능

예시	설명
<code>f=open('file.txt', 'rt')</code>	텍스트 읽기 모드로 파일을 연다. 기본값이며 rt는 생략 가능하다.
<code>f=open('file.txt', 'wb')</code>	바이너리 쓰기 모드로 파일을 연다.
<code>f=open('file.txt', 'rt+')</code>	텍스트 읽기 쓰기 모드로 기존 파일의 맨 뒤에서부터 내용을 쓴다.
<code>f=open('file.txt', 'wt+')</code>	텍스트 읽기 쓰기 모드로 기존 파일의 내용을 지우고 처음부터 쓴다.



1. 파이썬 파일 처리(복습)

1.2 open 함수 모드 예시들

- open 모드에 +를 포함하여 기존 파일 내용을 읽고 쓰는 방식 변경 가능

✓ 실행 코드

```
f=open('file.txt', 'rt', encoding='UTF8')  
for line in f:  
    print(line, end="")
```

작은 일에 성실한 사람은 큰 일에도 성실하고,
작은 일에 정직하지 못한 사람은 큰 일에도 정직하지 못하다.
누가복음 16:10

변함없이 서로 사랑하는 이가 친구이며
위급할 때 서로 돕는 이가 형제이다.
잠언 17:17

1. 파이썬 파일 처리(복습)

1.3 자동으로 파일 열고 닫기: with 문

- 파일의 내용을 수정하거나 추가한 후에는 파일을 닫아 주어야 함
- open 함수에 with 문을 사용하면 파일 열고 닫기가 자동으로 처리 됨

```
with open(파일 이름, 모드) as 변수명:  
    처리문 블록
```

✓ 실행 코드

```
with open("proverb.txt", 'r', encoding='UTF8', newline='') as filereader:  
    for row in filereader:  
        print("{}".format(row.strip( )))
```

작은 일에 성실한 사람은 큰 일에도 성실하고,
작은 일에 정직하지 못한 사람은 큰 일에도 정직하지 못하다.
누가복음 16:10

변함없이 서로 사랑하는 이가 친구이며
위급할 때 서로 돕는 이가 형제이다.
잠언 17:17

1. 파이썬 파일 처리(복습)

1.4 여러 파일읽기: os와 glob 모듈

- 폴더 내의 텍스트 파일 여러 개를 모두 읽어 오려면 os와 glob 모듈 импорт

✓ 실행 코드

```
import os
import glob
for file in glob.glob(os.path.join('*.*txt')):
    with open(file, 'r', newline='', encoding='UTF8') as filereader:
        for row in filereader:
            print("{}".format(row.strip( )))
```

작은 일에 성실한 사람은 큰 일에도 성실하고, 작은 일에 정직하지 못한 사람은 큰 일에도 정직하지 못하다.
누가복음 16:10

변함없이 서로 사랑하는 이가 친구이며
위급할 때 서로 돕는 이가 형제이다.

잠언 17:17

안녕하세요

hello

내용을 추가합니다.

저장하는 문서 이름, 'w' 대신 'a'를 사용합니다.

작은 일에 성실한 사람은 큰 일에도 성실하고,

작은 일에 정직하지 못한 사람은 큰 일에도 정직하지 못하다.

누가복음 16:10

변함없이 서로 사랑하는 이가 친구이며
위급할 때 서로 돕는 이가 형제이다.

잠언 17:17

1. 파이썬 파일 처리(복습)

1.5 문자열 공백/문자 제거: strip, lstrip, rstrip 함수

- strip 함수는 문자열 중간 공백은 제거되지 않으며 양쪽에 있는 공백 제거
- lstrip 함수는 문자열 기준으로 왼쪽 공백이나 지정된 문자 제거
- rstrip 함수는 문자열 기준으로 오른쪽 공백이나 지정된 문자 제거

✓ 실행 코드

```
b1=" 작은 일에 성실한 사람은          큰일에도 성실하고 "
```

```
b1.strip( ) # 공백 제거
```

```
text="....작은 일에 성실한 사람은, 큰일에도 성실하고...."
```

```
print(text.lstrip('.'))
```

```
print(text.rstrip('.'))
```

```
print(text.strip('.'))
```

```
작은 일에 성실한 사람은, 큰일에도 성실하고....  
....작은 일에 성실한 사람은, 큰일에도 성실하고  
작은 일에 성실한 사람은, 큰일에도 성실하고
```

1. 파이썬 파일 처리(복습)

1.5 문자열 분리: split 함수

✓ 실행 코드

```
text="....작은 일에 성실한 사람은, 큰일에도 성실하고...."  
print(text.split(", "))
```

```
['....작은 일에 성실한 사람은', '큰일에도 성실하고....']
```

✓ 실행 코드

```
numbers="1. 2. 3. 4. 5. 6"  
result=numbers.split(". ")  
print(result)
```

```
['1', '2', '3', '4', '5', '6']
```


1. 파이썬 파일 처리(복습)

1.6 판다스에서 사용 가능한 파일 종류, 읽기 및 저장 함수

[파일 종류]

- CSV(Comma Separated Values) 파일: 데이터를 쉼표로 구분해놓은 텍스트 파일
- 엑셀 파일

[파일 읽기 함수]

- CSV 파일: `pd.read_csv('파일명')`
- 엑셀 파일: `pd.read_excel('파일명')`

[파일 저장 함수]

- CSV 파일: `pd.to_csv('파일명')`
- 엑셀 파일: `pd.to_excel('파일명')`

1. 파이썬 파일 처리(복습)

1.7 웹 페이지에서 CSV 파일 가져오기

- 웹 페이지에서 CSV 파일을 가져와 테이터프레임으로 저장 `pd.read_csv('파일명')`

✓ 실행 코드

```
import pandas as pd
titanic_df=pd.read_csv('https://raw.githubusercontent.com/seh
akflower/data/main/titanic.csv')
titanic_df
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Braund	Mr.	Owen Harris	male	5	21171	7...		
2	1	1	Cumings	Mrs.	John Bradley (Florence Briggs Thayer)	female	1	9147	53.1	C	S
3	1	3	Heikkinen	Miss.	Laina	female	0	STON/O2. 31012	53.1		
4	1	1	Futrelle	Mrs.	Jacques Heath (Lily May Peel)	female	3	17367	53.1	C	S
5	0	3	Allen	Mr.	William Henry	male	3	17367	53.1		
...	...	...	...	...	...	...	...	...	...	...	...
152	1	1	Pears	Mrs.	Thomas (Edith Wearne)	female	2	17367	53.1	C	S
153	0	3	Meo	Mr.	Alfonzo	male	5	11206	8...		
154	0	3	van Billiard	Mr.	Austin Blyler	male	4	0.5	851...		
155	0	3	Olsen	Mr.	Ole Martin	male	0	265302	7.31...		
156	0	1	Williams	Mr.	Charles Duane	male	5	17597	t...		

156 rows × 12 columns

1. 파이썬 파일 처리(복습)

1.8 탭을 기준으로 분리하기

- CSV 파일을 불러올 때 옵션 `sep=' \t'`를 이용하여 탭 기준으로 열을 구분

✓ 실행 코드

```
url='https://raw.githubusercontent.com/sehakflower/data/main/titanic.csv'
titanic_df=pd.read_csv(url, sep=' \t')
titanic_df
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	
151	152	1	1	Pears, Mrs. Thomas (Edith Wearne)	female	22.0	1	0	113776	66.6000	C2	S
152	153	0	3	Meo, Mr. Alfonzo	male	55.5	0	0	A.5. 11206	8.0500	NaN	S
153	154	0	3	van Billiard, Mr. Austin Blyler	male	40.5	0	2	A/5. 851	14.5000	NaN	S
154	155	0	3	Olsen, Mr. Ole Martin	male	NaN	0	0	Fa 265302	7.3125	NaN	S
155	156	0	1	Williams, Mr. Charles Duane	male	51.0	0	1	PC 17597	61.3792	NaN	C

156 rows × 12 columns

1. 파이썬 파일 처리(복습)

1.9 데이터 분리하기

- 웹 sep="를 이용하면 공백이나 특수문자를 기준으로 데이터 분리 가능

✓ 실행 코드

```
print('apple', 'banana')
print('apple', 'banana', sep='')
print('apple', 'banana', sep=' ')
print('apple', 'banana', sep=' ')
print('apple', 'banana', sep=' -> -> ')
print('apple', 'banana', 'eggs', 'ham')
print('apple', 'banana', 'eggs', 'ham', sep='\t')
print('apple', 'banana', 'eggs', 'ham', sep='\t\t')
```

```
apple banana
applebanana
apple banana
apple banana
apple -> -> banana
apple banana eggs ham
apple      banana    eggs      ham
apple           banana          eggs          ham
```

1. 파이썬 파일 처리(복습)

1.10 엑셀 파일 가져오기

- 판다스를 통해 엑셀 파일 가져오기 가능
- 특히 엑셀 파일의 특정 시트Sheet 지정해서 가져오기 가능
- 별도의 옵션을 설정하지 않으면 가장 첫 번째 시트를 데이터프레임으로 읽음

✓ 실행 코드

```
titanic='https://github.com/sehakflower/data/blob/main/titanic.xlsx?raw=true'  
titanic=pd.read_excel(titanic,sheet_name='titanic3')  
titanic
```

	pclass	survived	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked	boat	body	home.dest
0	1	1	Allen, Miss. Elisabeth Walton	female	29.0000	0	0	24160	211.3375	B5	S	2	NaN	St Louis, MO
1	1	1	Allison, Master. Hudson Trevor	male	0.9167	1	2	113781	151.5500	C22 C26	S	11	NaN	Montreal, PQ / Chesterville, ON
2	1	0	Allison, Miss. Helen Loraine	female	2.0000	1	2	113781	151.5500	C22 C26	S	NaN	NaN	Montreal, PQ / Chesterville, ON
3	1	0	Allison, Mr. Hudson Joshua Creighton	male	30.0000	1	2	113781	151.5500	C22 C26	S	NaN	135.0	Montreal, PQ / Chesterville, ON
4	1	0	Allison, Mrs. Hudson J C (Bessie Waldo Daniels)	female	25.0000	1	2	113781	151.5500	C22 C26	S	NaN	NaN	Montreal, PQ / Chesterville, ON
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1304	3	0	Zabour, Miss. Hileni	female	14.5000	1	0	2665	14.4542	NaN	C	NaN	328.0	NaN
1305	3	0	Zabour, Miss. Thamine	female	NaN	1	0	2665	14.4542	NaN	C	NaN	NaN	NaN
1306	3	0	Zakarian, Mr. Mapriededer	male	26.5000	0	0	2656	7.2250	NaN	C	NaN	304.0	NaN
1307	3	0	Zakarian, Mr. Ortin	male	27.0000	0	0	2670	7.2250	NaN	C	NaN	NaN	NaN
1308	3	0	Zimmerman, Mr. Leo	male	29.0000	0	0	315082	7.8750	NaN	S	NaN	NaN	NaN

1309 rows × 14 columns

2. 파이썬 데이터 다루기

2.1 앞뒤 데이터 보기: head/tail 함수

- head 함수는 데이터프레임 초기 5개 행 출력
 - 인자값을 입력하지 않으면 인덱스 0부터 4까지 5행의 데이터 출력
- tail 함수는 head 함수와 반대로 맨 끝 5개 행의 데이터 출력
 - 인자값을 입력하지 않으면 인덱스 0부터 4까지 5행의 데이터 출력
- 두 함수 모두 괄호 안에 원하는 행의 수를 입력하여 원하는 만큼 출력 가능

✓ 실행 코드

```
titanic_df.head(7)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S

```
titanic_df.tail(7)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
149	150	0	2	Byles, Rev. Thomas Roussel Davids	male	42.0	0	0	244310	13.0000	NaN	S
150	151	0	2	Bateman, Rev. Robert James	male	51.0	0	0	S.O.P. 1166	12.5250	NaN	S
151	152	1	1	Pears, Mrs. Thomas (Edith Wearne)	female	22.0	1	0	113776	66.6000	C2	S
152	153	0	3	Meo, Mr. Alfonzo	male	55.5	0	0	A.5. 11206	8.0500	NaN	S
153	154	0	3	van Billiard, Mr. Austin Blyler	male	40.5	0	2	A/5. 851	14.5000	NaN	S
154	155	0	3	Olsen, Mr. Ole Martin	male	NaN	0	0	Fa 265302	7.3125	NaN	S
155	156	0	1	Williams, Mr. Charles Duane	male	51.0	0	1	PC 17597	61.3792	NaN	C

2. 파이썬 데이터 다루기

2.2 데이터 개수/형태 확인하기: len/shape 함수

- len 함수를 사용하면 데이터의 개수 확인 가능 len(데이터프레임 이름)
- shape 함수는 데이터프레임의 형태를 알아보기 위해 사용
- 결과는 tuple 형태로 반환

✓ 실행 코드

```
len(titanic_df)
```

156

PassengerId	Survived	Pclass	Name Sex Age SibSp Parch Ticket Fare Cabin Embarked														
0	1	0	3														
1	2	1	1	Cumings, Mrs. Joh	149	150	0	2	Bytes, Rev. Thomas Rousset Davids	male	42.0	0	0	244310	13.0000	NaN	S
2	3	1	3		150	151	0	2	Bateman, Rev. Robert James	male	51.0	0	0	S.O.P. 1166	12.5250	NaN	S
3	4	1	1	Futrelle, Mrs	151	152	1	1	Pears, Mrs. Thomas (Edith Wearne)	female	22.0	1	0	113776	66.6000	C2	S
4	5	0	3		152	153	0	3	Meo, Mr. Alfonzo	male	55.5	0	0	A/5. 11206	8.0500	NaN	S
5	6	0	3		153	154	0	3	van Billiard, Mr. Austin Blyler	male	40.5	0	2	A/5. 851	14.5000	NaN	S
6	7	0	1		154	155	0	3	Olsen, Mr. Ole Martin	male	NaN	0	0	Fa 265302	7.3125	NaN	S
					155	156	0	1	Williams, Mr. Charles Duane	male	51.0	0	1	PC 17597	61.3792	NaN	C

```
titanic_df.shape
```

(156, 12)

2. 파이썬 데이터 다루기

2.3 데이터의 기본 통계치 보기: describe 함수

- describe 함수를 사용하여 데이터의 기본 통계치 확인 가능
- 데이터형이 float나 int인 열에만 적용
- 각 열의 개수(count), 평균(mean), 표준편차(std), 최솟값(min), 최댓값(max) 등의 통계치 확인 가능

✓ 실행 코드

```
titanic_df.describe( )
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	156.000000	156.000000	156.000000	126.000000	156.000000	156.000000	156.000000
mean	78.500000	0.346154	2.423077	28.141508	0.615385	0.397436	28.109587
std	45.177428	0.477275	0.795459	14.613880	1.056235	0.870146	39.401047
min	1.000000	0.000000	1.000000	0.830000	0.000000	0.000000	6.750000
25%	39.750000	0.000000	2.000000	19.000000	0.000000	0.000000	8.003150
50%	78.500000	0.000000	3.000000	26.000000	0.000000	0.000000	14.454200
75%	117.250000	1.000000	3.000000	35.000000	1.000000	0.000000	30.371850
max	156.000000	1.000000	3.000000	71.000000	5.000000	5.000000	263.000000

2. 파이썬 데이터 다루기

2.4 결측치와 데이터형 확인하기: info 함수

- info 함수를 사용하여 데이터프레임 각 열의 결측치 여부와 데이터형 출력

✓ 실행 코드

```
titanic_df.info( )
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 156 entries, 0 to 155
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  156 non-null   int64
1   Survived     156 non-null   int64
2   Pclass       156 non-null   int64
3   Name         156 non-null   object
4   Sex          156 non-null   object
5   Age         126 non-null   float64
6   SibSp        156 non-null   int64
7   Parch        156 non-null   int64
8   Ticket       156 non-null   object
9   Fare         156 non-null   float64
10  Cabin        31 non-null    object
11  Embarked     155 non-null   object
dtypes: float64(2), int64(5), object(5)
memory usage: 14.8+ KB
```

2. 파이썬 데이터 다루기

2.5 열 이름 변경하기

- 판다스에서 제공하는 속성과 함수 사용
- `columns` 속성을 사용하여 데이터프레임이 갖고 있는 열 이름 확인 가능
 - `dtype`이 `object`인 것은 문자열 의미
- `columns` 속성을 사용하여 열 이름 한꺼번에 변경 가능
 - 열 개수가 정확하게 일치해야 함

✓ 실행 코드

```
titanic_df.columns
```

```
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',  
      'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],  
      dtype='object')
```

2. 파이썬 데이터 다루기

2.5 열 이름 변경하기

✓ 실행 코드

```
new_columns=['passengerId', 'survived', 'pclass', 'name', 'sex', 'age', 'sibsp', 'parch',  
            'ticket', 'fare', 'cabin', 'embarked']  
  
titanic_df.columns=new_columns  
  
titanic_df.columns
```

```
Index(['passengerId', 'survived', 'pclass', 'name', 'sex', 'age', 'sibsp',  
      'parch', 'ticket', 'fare', 'cabin', 'embarked'],  
      dtype='object')
```

✓ 실행 코드

```
titanic_df
```

	passengerId	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C

2. 파이썬 데이터 다루기

2.6 데이터프레임 복사하기

- 데이터프레임에 행/열을 추가하는 실습 전에 복사본 만들어 둘 것을 권장

✓ 실행 코드

```
copy_titanic_df=titanic_df.copy( )  
copy_titanic_df.head( )
```

	passengerId	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

2. 파이썬 데이터 다루기

2.7 행 추가하기

- 데이터를 딕셔너리 형태로 만든 뒤 `append` 함수 사용
- `'ignore_index=True'` 옵션을 추가하여 오류 방지
- `append` 함수로 데이터 추가 뒤 다시 `copy_titanic_df` 변수에 저장해야 변경된 값 유지

✓ 실행 코드

```
copy_titanic_df = pd.concat([copy_titanic_df, pd.DataFrame([{'passengerId':1,
'survived':1, 'pclass':1, 'name':1, 'sex':1,
'age':1, 'sibsp':1, 'parch':1, 'ticket':1, 'fare':1, 'cabin':1,
'embarked':1}])], ignore_index=True)
```

```
copy_titanic_df.tail( )
```

	passengerId	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
152	153	0	3	Meo, Mr. Alfonzo	male	55.5	0	0	A.5. 11206	8.0500	NaN	S
153	154	0	3	van Billiard, Mr. Austin Blyler	male	40.5	0	2	A/5. 851	14.5000	NaN	S
154	155	0	3	Olsen, Mr. Ole Martin	male	NaN	0	0	Fa 265302	7.3125	NaN	S
155	156	0	1	Williams, Mr. Charles Duane	male	51.0	0	1	PC 17597	61.3792	NaN	C
156	1	1	1	1	1	1.0	1	1	1	1.0000	1	1

2. 파이썬 데이터 다루기

2.8 열 추가하기

- `copy_titanic_df['추가할 열 이름']`
- `loc` 함수 사용하여 값 변경
- `head` 함수 사용하여 변경된 값 확인

✓ 실행 코드

```
copy_titanic_df['gender']=1  
copy_titanic_df.tail( )
```

	passengerId	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked	gender
154	155	0	3	Olsen, Mr. Ole Martin	male	NaN	0	0	Fa 265302	7.3125	NaN	S	1
155	156	0	1	Williams, Mr. Charles Duane	male	51.0	0	1	PC 17597	61.3792	NaN	C	1
156	1	1	1		1	1	1.0	1	1	1.0000	1	1	1

2. 파이썬 데이터 다루기

2.8 열 추가하기

- `copy_titanic_df['추가할 열 이름']`
- `loc` 함수 사용하여 값 변경
- `head` 함수 사용하여 변경된 값 확인

✓ 실행 코드

```
copy_titanic_df.loc[copy_titanic_df['sex']=='female', 'gender']=0  
  
copy_titanic_df.head( )
```

	passengerId	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked	gender
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S	1
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C	0
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S	0
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S	0
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S	1

2. 파이썬 데이터 다루기

2.9 index 속성

- 데이터프레임이 갖는 행의 시작 번호와 끝 번호의 증가값 확인 가능 `titanic_df.index`

```
titanic_df.index
```

```
RangeIndex(start=0, stop=156, step=1)
```

2.10 values 속성

- 인덱스를 제외한 값만 출력할 때 사용 `titanic_df.values` 를 실행하면 배열형태로 인덱스 표시

```
titanic_df.values
```

```
array([[1, 0, 3, ..., 7.25, nan, 'S'],  
       [2, 1, 1, ..., 71.2833, 'C85', 'C'],  
       [3, 1, 3, ..., 7.925, nan, 'S'],  
       ...,  
       [154, 0, 3, ..., 14.5, nan, 'S'],  
       [155, 0, 3, ..., 7.3125, nan, 'S'],  
       [156, 0, 1, ..., 61.3792, nan, 'C']], dtype=object)
```


2. 파이썬 데이터 다루기

2.11 열 추출하기

- 데이터프레임에서 특정 열만 추출하여 분리 가능

✓ 실행 코드

```
age=titanic_df['age'] # Series 객체는 1차원 배열(ndarray)과 호환됨

print(type(age))

age
```

```
<class 'pandas.core.series.Series'>
0      22.0
1      38.0
2      26.0
3      35.0
4      35.0
...
151     22.0
152     55.5
153     40.5
154      NaN
155     51.0
Name: age, Length: 156, dtype: float64
```

2. 파이썬 데이터 다루기

2.12 인덱스 기준으로 정렬하기

- `sort_index` 함수 사용
- `ascending` 옵션을 `True`로 지정하거나 생략하면 오름차순, `False`로 지정하면 내림차순으로 정렬

✓ 실행 코드

```
titanic_df.sort_index( )
```

	passengerId	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
151	152	1	1	Pears, Mrs. Thomas (Edith Wearne)	female	22.0	1	0	113776	66.6000	C2	S
152	153	0	3	Meo, Mr. Alfonzo	male	55.5	0	0	A.5. 11206	8.0500	NaN	S
153	154	0	3	van Billiard, Mr. Austin Blyler	male	40.5	0	2	A/5. 851	14.5000	NaN	S
154	155	0	3	Olsen, Mr. Ole Martin	male	NaN	0	0	Fa 265302	7.3125	NaN	S
155	156	0	1	Williams, Mr. Charles Duane	male	51.0	0	1	PC 17597	61.3792	NaN	C

156 rows × 12 columns

2. 파이썬 데이터 다루기

2.13 열 기준으로 정렬하기

- 특정 열을 기준으로 정렬할 때 `sort_values(by='age')`와 같이 설정
- `ascending` 옵션을 `True`로 지정하거나 생략하면 오름차순, `False`로 지정하면 내림차순으로 정렬
- 두 개의 열을 기준으로 정렬하려면 `sort_values(by=['age', 'pclass'])`로 설정
 - `ascending` 옵션으로 정렬 순서를 정함

✓ 실행 코드

```
titanic_df.sort_values(by='age') # 오름차순 정렬  
titanic_df.sort_values(by='age', ascending=False) # 내림차순 정렬
```

	passengerId	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
96	97	0	1	Goldschmidt, Mr. George B	male	71.0	0	0	PC 17754	34.6542	A5	C
116	117	0	3	Connors, Mr. Patrick	male	70.5	0	0	370369	7.7500	NaN	Q
33	34	0	2	Wheadon, Mr. Edward H	male	66.0	0	0	C.A. 24579	10.5000	NaN	S
54	55	0	1	Ostby, Mr. Engelhart Cornelius	male	65.0	0	1	113509	61.9792	B30	C
94	95	0	3	Coxon, Mr. Daniel	male	59.0	0	0	364500	7.2500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
121	122	0	3	Moore, Mr. Leonard Charles	male	NaN	0	0	A4. 54510	8.0500	NaN	S
126	127	0	3	McMahon, Mr. Martin	male	NaN	0	0	370372	7.7500	NaN	Q
128	129	1	3	Peter, Miss. Anna	female	NaN	1	1	2668	22.3583	F E69	C
140	141	0	3	Boulos, Mrs. Joseph (Sultana)	female	NaN	0	2	2678	15.2458	NaN	C
154	155	0	3	Olsen, Mr. Ole Martin	male	NaN	0	0	Fa 265302	7.3125	NaN	S

156 rows × 12 columns

2. 파이썬 데이터 다루기

2.13 열 기준으로 정렬하기

- 특정 열을 기준으로 정렬할 때 `sort_values(by='age')`와 같이 설정
- `ascending` 옵션을 `True`로 지정하거나 생략하면 오름차순, `False`로 지정하면 내림차순으로 정렬
- 두 개의 열을 기준으로 정렬하려면 `sort_values(by=['age', 'pclass'])`로 설정
 - `ascending` 옵션으로 정렬 순서를 정함

✓ 실행 코드

```
titanic_df.sort_values(by=['age', 'pclass']) # 오름차순 정렬
titanic_df.sort_values(by=['age', 'pclass'], ascending=False) # 내림차순 정렬
```

	passengerId	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
96	97	0	1	Goldschmidt, Mr. George B	male	71.0	0	0	PC 17754	34.6542	A5	C
116	117	0	3	Connors, Mr. Patrick	male	70.5	0	0	370369	7.7500	NaN	Q
33	34	0	2	Wheadon, Mr. Edward H	male	66.0	0	0	C.A. 24579	10.5000	NaN	S
54	55	0	1	Ostby, Mr. Engelhart Cornelius	male	65.0	0	1	113509	61.9792	B30	C
94	95	0	3	Coxon, Mr. Daniel	male	59.0	0	0	364500	7.2500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
154	155	0	3	Olsen, Mr. Ole Martin	male	NaN	0	0	Fa 265302	7.3125	NaN	S
17	18	1	2	Williams, Mr. Charles Eugene	male	NaN	0	0	244373	13.0000	NaN	S
31	32	1	1	Spencer, Mrs. William Augustus (Marie Eugenie)	female	NaN	1	0	PC 17569	146.5208	B78	C
55	56	1	1	Woolner, Mr. Hugh	male	NaN	0	0	19947	35.5000	C52	S
64	65	0	1	Stewart, Mr. Albert A	male	NaN	0	0	PC 17605	27.7208	NaN	C

156 rows × 12 columns

2. 파이썬 데이터 다루기

2.14 지정 범위의 행에서 데이터 가져오기

- 행을 슬라이싱하는 방법으로 원하는 영역까지 표시 가능
- `titanic_df[x:y]` 형태로 설정(x번째부터 y번째까지 행 추출)
- `titanic_df[::-1]`로 설정하면 행의 순서를 거꾸로 변경 가능
- 각각의 설정이 어떻게 적용되는지에 대한 확인은, 관련 없는 코드 앞에 주석 표시(#)을 추가하고 확인하고자 하는 코드만 실행

2. 파이썬 데이터 다루기

2.14 지정 범위의 행에서 데이터 가져오기

✓ 실행 코드

```
titanic_df[0:5] # titanic_df.head()와 실행 결과 동일함  
titanic_df[0:1] # 첫 행만 가져오기  
titanic_df[::3] # 3개씩 건너서 출력함  
titanic_df[::-1] # 행의 순서를 거꾸로 출력함
```

	passengerId	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
155	156	0	1	Williams, Mr. Charles Duane	male	51.0	0	1	PC 17597	61.3792	NaN	C
154	155	0	3	Olsen, Mr. Ole Martin	male	NaN	0	0	Fa 265302	7.3125	NaN	S
153	154	0	3	van Billiard, Mr. Austin Blyler	male	40.5	0	2	A/5. 851	14.5000	NaN	S
152	153	0	3	Meo, Mr. Alfonzo	male	55.5	0	0	A.5. 11206	8.0500	NaN	S
151	152	1	1	Pears, Mrs. Thomas (Edith Wearne)	female	22.0	1	0	113776	66.6000	C2	S
...	...	...	...	...	...	...	...	...	...	...	...	...
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S

156 rows × 12 columns

2. 파이썬 데이터 다루기

2.15 단순 인덱스로 범위 지정해 데이터 가져오기

- 단순 인덱스를 사용하여 범위를 선택
- 예) `titanic_df[:3]`
- `[:3]`으로 슬라이싱하면 첫 번째부터 세 번째 데이터까지 지정한 것으로, 0부터 2까지 세 개의 인덱스 값 데이터가 나타남

✓ 실행 코드

```
titanic_df[:3]
```

	passengerId	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S

2. 파이썬 데이터 다루기

2.16 지정 범위의 열에서 데이터 가져오기

- 열의 범위를 지정해서 데이터 가져오기 가능

- ① 먼저, `len` 함수와 `columns` 속성을 사용해서 데이터프레임 열의 개수 확인
- ② `columns` 속성을 `titanic_df[df.columns[열 번호]]` 형태로 설정
 - 해당하는 열 이름과 인덱스를 함께 나타냄
 - 열 번호는 열의 순서이며 0은 첫 번째 열을 의미
 - 마지막 열 데이터는 열 번호 직접 입력 또는 -1로 설정

✓ 실행 코드

```
len(titanic_df.columns)
```

```
12
```


2. 파이썬 데이터 다루기

2.16 지정 범위의 열에서 데이터 가져오기

✓ 실행 코드

```
titanic_df[titanic_df.columns[0]] # 첫 번째 (0번) 열을 1차원 배열의  
Series 객체로 추출
```

```
0    1  
1    2  
2    3  
3    4  
4    5
```

...

```
151 152  
152 153  
153 154  
154 155  
155 156
```

```
Name: passengerId, Length: 156, dtype: int64
```

2. 파이썬 데이터 다루기

2.16 지정 범위의 열에서 데이터 가져오기

✓ 실행 코드

```
titanic_df[titanic_df.columns[11]] # 마지막 열을 1차원 배열의 Series  
객체로 추출  
  
# titanic_df[titanic_df.columns[-1]] # 혹은 [-1]로 표시
```

```
0    S  
1    C  
2    S  
3    S  
4    S  
..  
151  S  
152  S  
153  S  
154  S  
155  C  
Name: embarked, Length: 156, dtype: object
```

2. 파이썬 데이터 다루기

2.17 열 이름으로 데이터 가져오기

- age 변수에 데이터프레임['열 이름'] 형태로 데이터 할당하고 print 함수에 type 함수로 열을 출력하면 Series라고 나타남
 - Series 객체는 1차원 배열 형태이며 ndarray와 호환 됨
- 데이터프레임 이름['column 이름'] 형태로 입력하면 특정 column의 데이터를 Series 객체로 추출하여 표시

✓ 실행 코드

```
age=titanic_df['age']  
print(type(age))
```

```
<class 'pandas.core.series.Series'>
```

✓ 실행 코드

```
titanic_df['age'] # 'age' 열을 1차원 배열인 Series 객체로 추출  
titanic_df.age
```

```
0    22.0  
1    38.0  
2    26.0  
3    35.0  
4    35.0
```

```
...
```

```
151   22.0  
152   55.5  
153   40.5  
154    NaN  
155   51.0
```

```
Name: age, Length: 156, dtype: float64
```

2. 파이썬 데이터 다루기

2.18 열 이름과 위치를 이용해 데이터 가져오기

- 인덱싱 사용하여 특정 범위의 데이터 가져올 수 있음
- [시작값:종료값:증가값] 형태로 입력
 - 시작값과 종료값 생략 가능하지만 콜론은 반드시 입력해야 함
 - 증가값을 생략하면 1씩 증가
- 여러 개의 열 이름을 리스트 형태로 만들어 특정 열을 가져올 수 있음
- 열 순서(열 번호)를 사용하여 데이터 추출 가능
- `columns` 속성을 사용하여 열 순서를 기준으로 데이터 가져올 수 있음

2. 파이썬 데이터 다루기

2.18 열 이름과 위치를 이용해 데이터 가져오기

✓ 실행 코드

```
print(titanic_df[titanic_df.columns[0]][0]) # 첫 번째 열의 첫 번째 값  
print(titanic_df['age'][0]) # age 열의 첫 번째 값  
print(titanic_df[titanic_df.columns[0]][2])  
print(titanic_df[titanic_df.columns[-1]][0])  
type(titanic_df[titanic_df.columns[0]][0]) # str
```

```
1  
22.0  
3  
S  
numpy.int64
```

2. 파이썬 데이터 다루기

2.18 열 이름과 위치를 이용해 데이터 가져오기

✓ 실행 코드

```
titanic_df[titanic_df.columns[0]][2:6] # 2번부터 5번 행까지  
titanic_df[titanic_df.columns[0]][:-1] # 마지막 행 제외  
titanic_df[titanic_df.columns[0]][::3] # 3개의 step으로 슬라이싱  
titanic_df[titanic_df.columns[0]][-1:] # 마지막 행만 추출  
titanic_df[titanic_df.columns[0]][::-1] # 행을 역순으로 출력
```

```
155 156  
154 155  
153 154  
152 153  
151 152
```

```
...  
4 5  
3 4  
2 3  
1 2  
0 1
```

```
Name: passengerId, Length: 156, dtype: int64
```

2. 파이썬 데이터 다루기

2.18 열 이름과 위치를 이용해 데이터 가져오기

✓ 실행 코드

```
titanic_df[['pclass', 'name', 'sex']] # DataFrame 추출  
titanic_df[[titanic_df.columns[2],titanic_df.columns[3],titanic_df  
.columns[4]]]  
# DataFrame 추출
```

	pclass	name	sex
0	3	Braund, Mr. Owen Harris	male
1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female
2	3	Heikkinen, Miss. Laina	female
3	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female
4	3	Allen, Mr. William Henry	male
...	...	...	...
151	1	Pears, Mrs. Thomas (Edith Wearne)	female
152	3	Meo, Mr. Alfonzo	male
153	3	van Billiard, Mr. Austin Blyler	male
154	3	Olsen, Mr. Ole Martin	male
155	1	Williams, Mr. Charles Duane	male

156 rows × 3 columns

2. 파이썬 데이터 다루기

2.18 열 이름과 위치를 이용해 데이터 가져오기

✓ 실행 코드

```
titanic_df[[titanic_df.columns[2],titanic_df.columns[3],titanic_d  
f.columns[4]]][2:15:3]  
# 슬라이싱
```

pclass		name	sex
2	3	Heikkinen, Miss. Laina	female
5	3	Moran, Mr. James	male
8	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female
11	1	Bonnell, Miss. Elizabeth	female
14	3	Vestrom, Miss. Hulda Amanda Adolfina	female

✓ 실행 코드

```
print(titanic_df[titanic_df.columns[0]][0])  
print(titanic_df[titanic_df.columns[0]][2])  
# print(titanic_df[titanic_df.columns[0]][-1]) # 실행 시 오류 발생  
type(titanic_df[titanic_df.columns[0]][0])
```

```
1  
3  
numpy.int64
```


2. 파이썬 데이터 다루기

2.19 인덱스 이름을 기준으로 데이터 가져오기: loc 속성

- 인덱스 이름을 기준으로 원하는 만큼의 행 데이터를 가져올 수 있음
- 기본 형식은 `titanic_df.loc[시작 인덱스 이름:종료 인덱스 이름:증가값]`
 - 행을 하나 또는 여러 개 지정할 수 있음
 - 0번부터 마지막 인덱스까지 모든 행을 가져올 수 있음
 - 특정 인덱스에서 필요한 열이나 열의 범위를 지정해서 가져올 수 있음
 - 증가 값을 설정하면 설정한 만큼 건너뛰어서 표시

✓ 실행 코드

```
titanic_df.loc[0]
```

```
passengerId      1
survived          0
pclass           3
name      Braund, Mr. Owen Harris
sex              male
age             22.0
sibsp            1
parch            0
ticket      A/5 21171
fare             7.25
cabin           NaN
embarked         S
Name: 0, dtype: object
```

2. 파이썬 데이터 다루기

2.19 인덱스 이름을 기준으로 데이터 가져오기: loc 속성

✓ 실행 코드

```
titanic_df.loc[0:5]
```

	passengerId	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q

✓ 실행 코드

```
titanic_df.loc[0:10:2]
```

	passengerId	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S
10	11	1	3	Sandstrom, Miss. Marguerite Rut	female	4.0	1	1	PP 9549	16.7000	G6	S

2. 파이썬 데이터 다루기

2.19 인덱스 이름을 기준으로 데이터 가져오기: loc 속성

✓ 실행 코드

```
titanic_df.loc[:, 'age']
```

```
0    22.0
```

```
1    38.0
```

```
2    26.0
```

```
3    35.0
```

```
4    35.0
```

```
...
```

```
151   22.0
```

```
152   55.5
```

```
153   40.5
```

```
154    NaN
```

```
155   51.0
```

```
Name: age, Length: 156, dtype: float64
```

2. 파이썬 데이터 다루기

2.19 인덱스 이름을 기준으로 데이터 가져오기: loc 속성

✓ 실행 코드

```
titanic_df.loc[:, ['age', 'sex']]
```

	age	sex
0	22.0	male
1	38.0	female
2	26.0	female
3	35.0	female
4	35.0	male
...	...	...
151	22.0	female
152	55.5	male
153	40.5	male
154	NaN	male
155	51.0	male

156 rows × 2 columns

2. 파이썬 데이터 다루기

2.19 인덱스 이름을 기준으로 데이터 가져오기: loc 속성

✓ 실행 코드

```
titanic_df.loc[3:7, ['age', 'sex']]
```

	age	sex
3	35.0	female
4	35.0	male
5	NaN	male
6	54.0	male
7	2.0	male

✓ 실행 코드

```
titanic_df.loc[2:5, 'survived':'age']
```

	survived	pclass	name	sex	age
2	1	3	Heikkinen, Miss. Laina	female	26.0
3	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0
4	0	3	Allen, Mr. William Henry	male	35.0
5	0	3	Moran, Mr. James	male	NaN

2. 파이썬 데이터 다루기

2.20 인덱스 순서를 기준으로 데이터 가져오기: iloc 속성

- 인덱스 순서를 기준으로 원하는 만큼의 행 데이터를 가져올 수 있음
- 기본 형식은 `titanic_df.loc[시작 인덱스 번호:종료 인덱스 번호:증가값]`
 - 인덱스 번호만 입력하면 한 행의 데이터만 표시
 - 행 범위를 :으로 표현하면 표현한 모든 범위의 행을 가져올 수 있음
 - ,로 설정하면 설정한 열의 데이터만 가져올 수 있음
 - 증가 값 설정 가능

✓ 실행 코드

```
titanic_df.iloc[0]
```

```
passengerId      1
survived          0
pclass            3
name      Braund, Mr. Owen Harris
sex              male
age             22.0
sibsp             1
parch             0
ticket      A/5 21171
fare             7.25
cabin           NaN
embarked          S
Name: 0, dtype: object
```

2. 파이썬 데이터 다루기

2.20 인덱스 순서를 기준으로 데이터 가져오기: `iloc` 속성

✓ 실행 코드

```
titanic_df.iloc[0:5]
```

	passengerId	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

✓ 실행 코드

```
titanic_df.iloc[0:10:3]
```

	passengerId	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	NaN	C

2. 파이썬 데이터 다루기

2.20 인덱스 순서를 기준으로 데이터 가져오기: iloc 속성

✓ 실행 코드

```
titanic_df.iloc[:, [0, 2]]
```

	passengerId	pclass
0	1	3
1	2	1
2	3	3
3	4	1
4	5	3
...	...	...
151	152	1
152	153	3
153	154	3
154	155	3
155	156	1

156 rows × 2 columns

✓ 실행 코드

```
titanic_df.iloc[1:5, [0, 2]]
```

	passengerId	pclass
1	2	1
2	3	3
3	4	1
4	5	3

2. 파이썬 데이터 다루기

2.20 인덱스 순서를 기준으로 데이터 가져오기: `iloc` 속성

✓ 실행 코드

```
titanic_df.iloc[1:5, 1:4]
```

	survived	pclass	name
1	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...
2	1	3	Heikkinen, Miss. Laina
3	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)
4	0	3	Allen, Mr. William Henry

2. 파이썬 데이터 다루기

2.21 피벗 테이블을 이용해 데이터 가져오기

- `pivot_table` 함수

- ① 데이터프레임에 포함된 열 중 두 개의 열을 각각 행과 열 인덱스로 설정
- ② 그 행과 열이 만나는 지점에 있는 열의 데이터를 가져옴
 - 피벗 테이블 왼쪽 인덱스는 행 인덱스, 상단의 인덱스는 열 인덱스

✓ 실행 코드

```
titanic_df.pivot_table(index='pclass', columns='sex',  
values='survived')
```

	sex	female	male
pclass			
1		1.000000	0.142857
2		0.916667	0.166667
3		0.571429	0.131148



2. 파이썬 데이터 다루기

2.22 피벗 테이블을 이용해 데이터 가져오기

- `titanic_df.pivot_table(index='행 인덱스', columns='열 인덱스', values='계산에 사용할 열')`
 - `pivot_table`의 `aggfunc`에 `mean` 함수를 사용하면 `value`로 설정된 `survived` 열의 평균을 표시됨
 - `pivot_table`의 `aggfunc=np.sum`으로 설정하면 `value`로 설정된 `survived` 열의 합계가 표시됨

✓ 실행 코드

```
import numpy as np # mean 함수를 사용하기 위해 numpy를 import
titanic_df.pivot_table(index='pclass', columns='sex',
values='survived', aggfunc=np.mean)
```

	sex	female	male
pclass			
1		1.000000	0.142857
2		0.916667	0.166667
3		0.571429	0.131148

2. 파이썬 데이터 다루기

2.22 피벗 테이블을 이용해 데이터 가져오기

- `titanic_df.pivot_table(index='행 인덱스', columns='열 인덱스', values='계산에 사용할 열')`
 - `pivot_table`의 `aggfunc`에 `mean` 함수를 사용하면 `value`로 설정된 `survived` 열의 평균을 표시됨
 - `pivot_table`의 `aggfunc=np.sum`으로 설정하면 `value`로 설정된 `survived` 열의 합계가 표시됨

✓ 실행 코드

```
import numpy as np

titanic_df.pivot_table(index='pclass', columns='sex',
values='survived', aggfunc=np.sum)
```

	sex	female	male
pclass			
1		9	3
2		11	3
3		20	8

2. 파이썬 데이터 다루기

2.23 그룹으로 묶어서 데이터 가져오기

- `groupby` 함수 사용
- 데이터프레임의 특정 열을 기준으로 데이터를 묶어서 표시
 - `count`, `sum`, `mean` 함수를 함께 사용하여 그룹화된 데이터 개수, 합계, 평균 표시 가능
- 같은 데이터끼리 묶어서 분석하고 계산할 때 유용함
- 특정 열만 출력할 경우 ['열 이름']을 붙여 표시
- ['열 이름', '열 이름',...]과 같이 인덱스로 사용할 열 이름을 리스트 형태로 입력하면 행 인덱스를 두 개 이상의 열로 사용 가능

✓ 실행 코드

```
titanic_df.head( )
```

	passengerId	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

2. 파이썬 데이터 다루기

2.23 그룹으로 묶어서 데이터 가져오기

✓ 실행 코드

```
titanic_df.groupby('survived').count( )
```

	passengerId	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
survived											
0	102	102	102	102	85	102	102	102	102	14	102
1	54	54	54	54	41	54	54	54	54	17	53

✓ 실행 코드

```
titanic_df.groupby('survived').mean( )
```

	passengerId	pclass	age	sibsp	parch	fare
survived						
0	84.823529	2.490196	29.358824	0.666667	0.372549	27.271076
1	66.555556	2.296296	25.617805	0.518519	0.444444	29.693441

2. 파이썬 데이터 다루기

2.23 그룹으로 묶어서 데이터 가져오기

✓ 실행 코드

```
titanic_df.groupby('sex').sum( )
```

	passengerId	survived	pclass	age	sibsp	parch	fare
sex							
female	3815	40	138	1150.00	49	29	1593.7958
male	8431	14	240	2395.83	47	33	2791.2998

✓ 실행 코드

```
titanic_df.groupby('sex')['survived'].mean( )
```

```
sex
female    0.714286
male      0.140000
Name: survived, dtype: float64
```

2. 파이썬 데이터 다루기

2.23 그룹으로 묶어서 데이터 가져오기

✓ 실행 코드

```
titanic_df.groupby(['sex', 'survived']).mean( )
```

		passengerId	pclass	age	sibsp	parch	fare
sex	survived						
female	0	80.437500	2.937500	20.633333	1.437500	0.562500	18.883331
	1	63.200000	2.275000	26.265625	0.650000	0.500000	32.291563
male	0	85.639535	2.406977	31.228571	0.523256	0.337209	28.831587
	1	76.142857	2.357143	23.314444	0.142857	0.285714	22.270236

2. 파이썬 데이터 다루기

2.24 stack과 unstack

- **stack** 함수는 특정 열을 그룹화하여 행 단위로 표시하고 계산한 데이터를 나타냄
- **unstack** 함수는 열 단위로 표시하고 계산한 데이터를 나타냄
 - 지정한 열을 기준으로 모든 열 값을 풀어서 출력할 때 사용

[stack]

시도명	일자	값	
서귀포시	2018년	거주인구(명)	183437.5044
		방문인구(명)	136493.7039
	2019년	거주인구(명)	195625.3659
		방문인구(명)	142424.3788
	2020년	거주인구(명)	222758.3904
		방문인구(명)	117197.1859
제주시	2018년	거주인구(명)	400738.593
		방문인구(명)	237845.2388
	2019년	거주인구(명)	395796.5919
		방문인구(명)	244563.1351
	2020년	거주인구(명)	407192.6644
		방문인구(명)	205097.9053

[unstack]

시도명	일자	거주인구(명)	방문인구(명)
서귀포시	2018년	183437.5044	136493.7039
	2019년	195625.3659	142424.3788
	2020년	222758.3904	117197.1859
제주시	2018년	400738.593	237845.2388
	2019년	395796.5919	244563.1351
	2020년	407192.6644	205097.9053

stack

←·····

unstack

·····→

2. 파이썬 데이터 다루기

2.24 stack과 unstack

✓ 실행 코드

```
df = titanic_df.groupby(['sex', 'survived']).mean()  
df
```

		passengerId	pclass	age	sibsp	parch	fare
sex	survived						
female	0	80.437500	2.937500	20.633333	1.437500	0.562500	18.883331
	1	63.200000	2.275000	26.265625	0.650000	0.500000	32.291563
male	0	85.639535	2.406977	31.228571	0.523256	0.337209	28.831587
	1	76.142857	2.357143	23.314444	0.142857	0.285714	22.270236

2. 파이썬 데이터 다루기

2.24 stack과 unstack

✓ 실행 코드

```
df.unstack('sex')
```

	passengerId		pclass		age		sibsp		parch		fare	
sex	female	male	female	male	female	male	female	male	female	male	female	male
survived												
0	80.4375	85.639535	2.9375	2.406977	20.633333	31.228571	1.4375	0.523256	0.5625	0.337209	18.883331	28.831587
1	63.2000	76.142857	2.2750	2.357143	26.265625	23.314444	0.6500	0.142857	0.5000	0.285714	32.291563	22.270236

✓ 실행 코드

```
df.unstack('survived')
```

	passengerId		pclass		age		sibsp		parch		fare	
survived	0	1	0	1	0	1	0	1	0	1	0	1
sex												
female	80.437500	63.200000	2.937500	2.275000	20.633333	26.265625	1.437500	0.650000	0.562500	0.500000	18.883331	32.291563
male	85.639535	76.142857	2.406977	2.357143	31.228571	23.314444	0.523256	0.142857	0.337209	0.285714	28.831587	22.270236

2. 파이썬 데이터 다루기

2.24 stack과 unstack

✓ 실행 코드

```
survived=df.unstack('survived')  
survived_stack=survived.stack('survived')  
survived_stack
```

		passengerId	pclass	age	sibsp	parch	fare
sex	survived						
female	0	80.437500	2.937500	20.633333	1.437500	0.562500	18.883331
	1	63.200000	2.275000	26.265625	0.650000	0.500000	32.291563
male	0	85.639535	2.406977	31.228571	0.523256	0.337209	28.831587
	1	76.142857	2.357143	23.314444	0.142857	0.285714	22.270236

2. 파이썬 데이터 다루기

2.25 인덱스 초기화하기: reset_index 함수

- reset_index 함수는 멀티 인덱스로 구성된 데이터프레임의 인덱스 초기화에 사용
- 옵션을 inplace=True로 지정하면 데이터프레임명을 따로 지정하지 않고 원본 자체를 초기화 함

✓ 실행 코드

```
df1=df.reset_index( )  
df1
```

	sex	survived	passengerId	pclass	age	sibsp	parch	fare
0	female	0	80.437500	2.937500	20.633333	1.437500	0.562500	18.883331
1	female	1	63.200000	2.275000	26.265625	0.650000	0.500000	32.291563
2	male	0	85.639535	2.406977	31.228571	0.523256	0.337209	28.831587
3	male	1	76.142857	2.357143	23.314444	0.142857	0.285714	22.270236

2. 파이썬 데이터 다루기

2.25 인덱스 초기화하기: reset_index 함수

✓ 실행 코드

```
df.reset_index(inplace=True)  
df
```

	sex	survived	passengerId	pclass	age	sibsp	parch	fare
0	female	0	80.437500	2.937500	20.633333	1.437500	0.562500	18.883331
1	female	1	63.200000	2.275000	26.265625	0.650000	0.500000	32.291563
2	male	0	85.639535	2.406977	31.228571	0.523256	0.337209	28.831587
3	male	1	76.142857	2.357143	23.314444	0.142857	0.285714	22.270236

2. 파이썬 데이터 다루기

2.26 데이터 합치기: concat 함수

- concat 함수는 행의 인덱스와 열의 크기에 상관없이 위아래 혹은 옆으로 데이터를 합칠 때 사용
- 옵션을 지정하지 않으면 기본값으로 axis=0이 적용되어 위아래로 합치게 됨

✓ 실행 코드

```
import pandas as pd
url='https://github.com/sehakflower/data/blob/main/titanic_1309.xlsx?raw=true'
titanic=pd.read_excel(url, sheet_name='total')
titanic
```

	pclass	survived	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked	boat	body	home.dest
0	1	1	Allen, Miss. Elisabeth Walton	female	29.0000	0	0	24160	211.3375	B5	S	2	NaN	St Louis, MO
1	1	1	Allison, Master. Hudson Trevor	male	0.9167	1	2	113781	151.5500	C22 C26	S	11	NaN	Montreal, PQ / Chesterville, ON
2	1	0	Allison, Miss. Helen Loraine	female	2.0000	1	2	113781	151.5500	C22 C26	S	NaN	NaN	Montreal, PQ / Chesterville, ON
3	1	0	Allison, Mr. Hudson Joshua Creighton	male	30.0000	1	2	113781	151.5500	C22 C26	S	NaN	135.0	Montreal, PQ / Chesterville, ON
4	1	0	Allison, Mrs. Hudson J C (Bessie Waldo Daniels)	female	25.0000	1	2	113781	151.5500	C22 C26	S	NaN	NaN	Montreal, PQ / Chesterville, ON
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1304	3	0	Zabour, Miss. Hileni	female	14.5000	1	0	2665	14.4542	NaN	C	NaN	328.0	NaN
1305	3	0	Zabour, Miss. Thamine	female	NaN	1	0	2665	14.4542	NaN	C	NaN	NaN	NaN
1306	3	0	Zakarian, Mr. Mapriededer	male	26.5000	0	0	2656	7.2250	NaN	C	NaN	304.0	NaN
1307	3	0	Zakarian, Mr. Ortin	male	27.0000	0	0	2670	7.2250	NaN	C	NaN	NaN	NaN
1308	3	0	Zimmerman, Mr. Leo	male	29.0000	0	0	315082	7.8750	NaN	S	NaN	NaN	NaN

1309 rows × 14 columns

2. 파이썬 데이터 다루기

2.26 데이터 합치기: concat 함수

✓ 실행 코드

```
train_1000=titanic.iloc[:1000]  
test_309=titanic.iloc[1000:]  
len(train_1000), len(test_309)
```

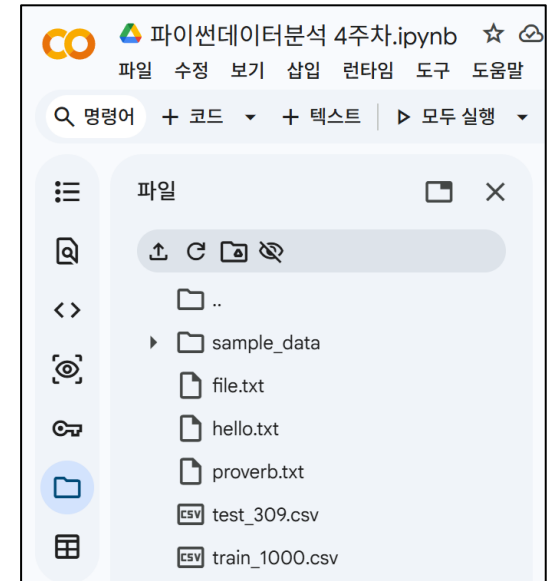
```
(1000, 309)
```

✓ 실행 코드

```
train_1000.to_csv('train_1000.csv')  
test_309.to_csv('test_309.csv')
```

```
pwd
```

```
/content
```



2. 파이썬 데이터 다루기

2.26 데이터 합치기: concat 함수

✓ 실행 코드

```
ls
```

```
file.txt hello.txt proverb.txt sample_data/ test_309.csv train_1000.csv
```

✓ 실행 코드

```
pd.read_csv('train_1000.csv')
```

Unnamed: 0	pclass	survived	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked	boat	body	home.dest	
0	0	1	1	Allen, Miss. Elisabeth Walton	female	29.0000	0	0	24160	211.3375	B5	S	2	NaN	St Louis, MO
1	1	1	1	Allison, Master. Hudson Trevor	male	0.9167	1	2	113781	151.5500	C22 C26	S	11	NaN	Montreal, PQ / Chesterville, ON
2	2	1	0	Allison, Miss. Helen Loraine	female	2.0000	1	2	113781	151.5500	C22 C26	S	NaN	NaN	Montreal, PQ / Chesterville, ON
3	3	1	0	Allison, Mr. Hudson Joshua Creighton	male	30.0000	1	2	113781	151.5500	C22 C26	S	NaN	135.0	Montreal, PQ / Chesterville, ON
4	4	1	0	Allison, Mrs. Hudson J C (Bessie Waldo Daniels)	female	25.0000	1	2	113781	151.5500	C22 C26	S	NaN	NaN	Montreal, PQ / Chesterville, ON
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
995	995	3	0	Markoff, Mr. Marin	male	35.0000	0	0	349213	7.8958	NaN	C	NaN	NaN	NaN
996	996	3	0	Markun, Mr. Johann	male	33.0000	0	0	349257	7.8958	NaN	S	NaN	NaN	NaN
997	997	3	1	Masselmani, Mrs. Fatima	female	NaN	0	0	2649	7.2250	NaN	C	C	NaN	NaN
998	998	3	0	Matinoff, Mr. Nicola	male	NaN	0	0	349255	7.8958	NaN	C	NaN	NaN	NaN
999	999	3	1	McCarthy, Miss. Catherine "Katie"	female	NaN	0	0	383123	7.7500	NaN	Q	15 16	NaN	NaN

1000 rows × 15 columns

2. 파이썬 데이터 다루기

2.26 데이터 합치기: concat 함수

✓ 실행 코드

```
total_df=pd.concat([train_1000, test_309]) # 옵션 없으면 axis=0

total_df=pd.concat([train_1000, test_309], axis=0) # 위아래로 붙일 때는 axis=0

total_df
```

	pclass	survived	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked	boat	body	home.dest
0	1	1	Allen, Miss. Elisabeth Walton	female	29.0000	0	0	24160	211.3375	B5	S	2	NaN	St Louis, MO
1	1	1	Allison, Master. Hudson Trevor	male	0.9167	1	2	113781	151.5500	C22 C26	S	11	NaN	Montreal, PQ / Chesterville, ON
2	1	0	Allison, Miss. Helen Loraine	female	2.0000	1	2	113781	151.5500	C22 C26	S	NaN	NaN	Montreal, PQ / Chesterville, ON
3	1	0	Allison, Mr. Hudson Joshua Creighton	male	30.0000	1	2	113781	151.5500	C22 C26	S	NaN	135.0	Montreal, PQ / Chesterville, ON
4	1	0	Allison, Mrs. Hudson J C (Bessie Waldo Daniels)	female	25.0000	1	2	113781	151.5500	C22 C26	S	NaN	NaN	Montreal, PQ / Chesterville, ON
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1304	3	0	Zabour, Miss. Hileni	female	14.5000	1	0	2665	14.4542	NaN	C	NaN	328.0	NaN
1305	3	0	Zabour, Miss. Thamine	female	NaN	1	0	2665	14.4542	NaN	C	NaN	NaN	NaN
1306	3	0	Zakarian, Mr. Mapriededer	male	26.5000	0	0	2656	7.2250	NaN	C	NaN	304.0	NaN
1307	3	0	Zakarian, Mr. Ortin	male	27.0000	0	0	2670	7.2250	NaN	C	NaN	NaN	NaN
1308	3	0	Zimmerman, Mr. Leo	male	29.0000	0	0	315082	7.8750	NaN	S	NaN	NaN	NaN

1309 rows × 14 columns

2. 파이썬 데이터 다루기

2.26 데이터 합치기: concat 함수

✓ 실행 코드

```
total_df=pd.concat([train_1000, test_309])
total_df=pd.concat([train_1000, test_309], axis=1)
total_df
```

	pclass	survived	name	sex	age	sibsp	parch	ticket	fare	cabin	...	age	sibsp	parch	ticket	fare	cabin	embarked	boat	body	home.dest
0	1.0	1.0	Allen, Miss. Elisabeth Walton	female	29.0000	0.0	0.0	24160	211.3375	B5	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	1.0	1.0	Allison, Master. Hudson Trevor	male	0.9167	1.0	2.0	113781	151.5500	C22 C26	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	1.0	0.0	Allison, Miss. Helen Loraine	female	2.0000	1.0	2.0	113781	151.5500	C22 C26	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	1.0	0.0	Allison, Mr. Hudson Joshua Creighton	male	30.0000	1.0	2.0	113781	151.5500	C22 C26	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	1.0	0.0	Allison, Mrs. Hudson J C (Bessie Waldo Daniels)	female	25.0000	1.0	2.0	113781	151.5500	C22 C26	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1304	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	14.5	1.0	0.0	2665	14.4542	NaN	C	NaN	328.0	NaN
1305	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	1.0	0.0	2665	14.4542	NaN	C	NaN	NaN	NaN
1306	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	26.5	0.0	0.0	2656	7.2250	NaN	C	NaN	304.0	NaN
1307	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	27.0	0.0	0.0	2670	7.2250	NaN	C	NaN	NaN	NaN
1308	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	29.0	0.0	0.0	315082	7.8750	NaN	S	NaN	NaN	NaN

1309 rows × 28 columns



강남대학교
KANGNAM UNIVERSITY

감사합니다

